Comparación de modelos y su costo por request (Bloque 9) / Cómo elegir entre dos modelos candidatos, con criterio

✓ **Hecho:** Ver

Cómo elegir entre dos modelos candidatos, con criterio

En la lección anterior viste los tres factores que hacen que **"el mejor modelo"** no exista en abstract costo, latencia, y qué tan bien se ajusta a lo que tu caso específico necesita.

Esta lección convierte esos tres factores en un método concreto: una forma de comparar dos modelos candidatos con datos reales, para poder argumentar tu elección en vez de simplemente intuirla.

Prerrequisitos: No necesitas instalar nada nuevo: vas a usar el mismo módulo que ya tienes funcionando desde el Bloque 4, con tu archivo ".env" y tu credencial ya configurados. Sí vas a necesitar acceso a internet para consultar la página de precios vigente de cada modelo que compares.

📊 El método: una tabla, no una impresión

La forma más simple, y la más difícil de discutir, de comparar dos modelos es una tabla: los mismo criterios como filas, un modelo candidato por columna, y en cada celda un dato concreto, no una opinió Así se ve la estructura, con los datos todavía sin completar:

Criterio	Modelo A	Modelo B
Costo por 1000 tokens de entrada	valor real, de la página de precios del proveedor	valor real, de la página de precios del proveedor
Costo por 1000 tokens de salida	valor real, de la página de precios del proveedor	valor real, de la página de precios del proveedor
Latencia promedio observada	medida en tus propias pruebas, no estimada	medida en tus propias pruebas, no estimada
Calidad de respuesta para tu caso concreto	evaluada sobre preguntas reales de tu miniaplicación	evaluada sobre preguntas reales de tu miniaplicación

Los nombres **"Modelo A"** y **"Modelo B"** son genéricos a propósito: en tu propia tabla van los dos modelos reales que estés evaluando para tu miniaplicación, casi siempre dos variantes del mismo proveedor con el que ya vienes trabajando (una más económica y rápida, otra más capaz).

🔎 De dónde sale cada dato de la tabla



El costo por token

No se estima: se busca directamente en la página de precios vigente del proveedor para cada modelo. Es el único dato de la tabla que no generas tú mismo con una petición.



La latencia

Se mide, no se supone: cronometra el tiempo entre el momento en que tu función **"enviar_peticion"** envía la petición y el momento en que llega la respuesta, para cada modelo, con la misma pregunta de prueba.

La calidad de respuesta para tu caso

Se evalúa con dos o tres preguntas representativas de lo que tu miniaplicación realmente necesita responder, no con una impresión general de qué tan **"inteligente"** parece cada modelo en una conversación cualquiera.

La fecha en que armaste la tabla

Anotada junto a los otros datos. Tanto los precios como el comportamiento de los modelos cambian con el tiempo, y una tabla sin fecha pierde valor como evidencia apenas pasan unos meses: quien la lea después necesita saber si esos números todavía son válidos o si conviene volver a medirlos.

Sobre la latencia en particular, una sola medición por modelo no alcanza para ser representativa: el tiemp de respuesta varía de una petición a otra, incluso con la misma pregunta exacta, por motivos que n dependen de tu código, como el estado momentáneo del servicio o tu propia conexión de red.

Repite la prueba tres o cuatro veces por modelo y usa el promedio, no el primer resultado que obtenga para que la comparación sea justa.

Así se ve en código, siguiendo exactamente los pasos anteriores: repetir la prueba, guardar cada resultad y promediar al final. Usa la misma pregunta de prueba para las cuatro repeticiones, y el precio que y consultaste en el paso 1:

```
import time

PRECIO_ENTRADA = 0.00000010  # el valor real de la página de precios
PRECIO_SALIDA = 0.00000030

latencias = []
for i in range(4):
    url, body = armar_peticion(pregunta)
    inicio = time.perf_counter()
    respuesta_json = enviar_peticion(url, body)
    fin = time.perf_counter()
    latencias.append(fin - inicio)
    print(f"Intento {i + 1}: {latencias[-1]:.2f} segundos")

latencia_promedio = sum(latencias) / len(latencias)
print(f"Latencia promedio: {latencia_promedio:.2f} segundos")

costo = calcular_costo(respuesta_json, PRECIO_ENTRADA, PRECIO_SALIDA)
```

```
print(f"Costo de la última petición: {costo}")
```

Con la misma pregunta de prueba y los mismos números de ejemplo de las lecciones anteriores (85 tokens de entrada, 320 de salida), una salida de ejemplo se vería así:

```
Intento 1: 1.92 segundos
Intento 2: 1.79 segundos
Intento 3: 2.15 segundos
Intento 4: 1.86 segundos
Latencia promedio: 1.93 segundos
Costo de la última petición: 0.000181
```

Los tiempos exactos van a ser distintos en tu caso, y van a variar cada vez que corras el script: eso e justamente lo esperable. Lo que importa es el patrón (repetir, guardar cada resultado, promediar al final no estos números puntuales.



Troubleshooting

Repetir la petición varias veces seguidas hace más probable ver un error **429** (límite de uso) que con una sola petición: ya viste en la lección anterior qué significa y cómo manejarlo. Si ves un error de autenticación **(401)**, revisa lo mismo que ya armaste al cerrar el Bloque 4 para tu archivo **".env"**.

📊 **Sobre las mediciones:** si una de tus 3 o 4 repeticiones sale claramente distinta a las demás (por ejemplo, el doble de lenta), no la descartes automáticamente, pero tenlo presente: suele deberse a saturación momentánea del proveedor o de tu propia conexión, no a un problema en tu código. Si te pasa seguido, agrega una repetición más antes de promediar.

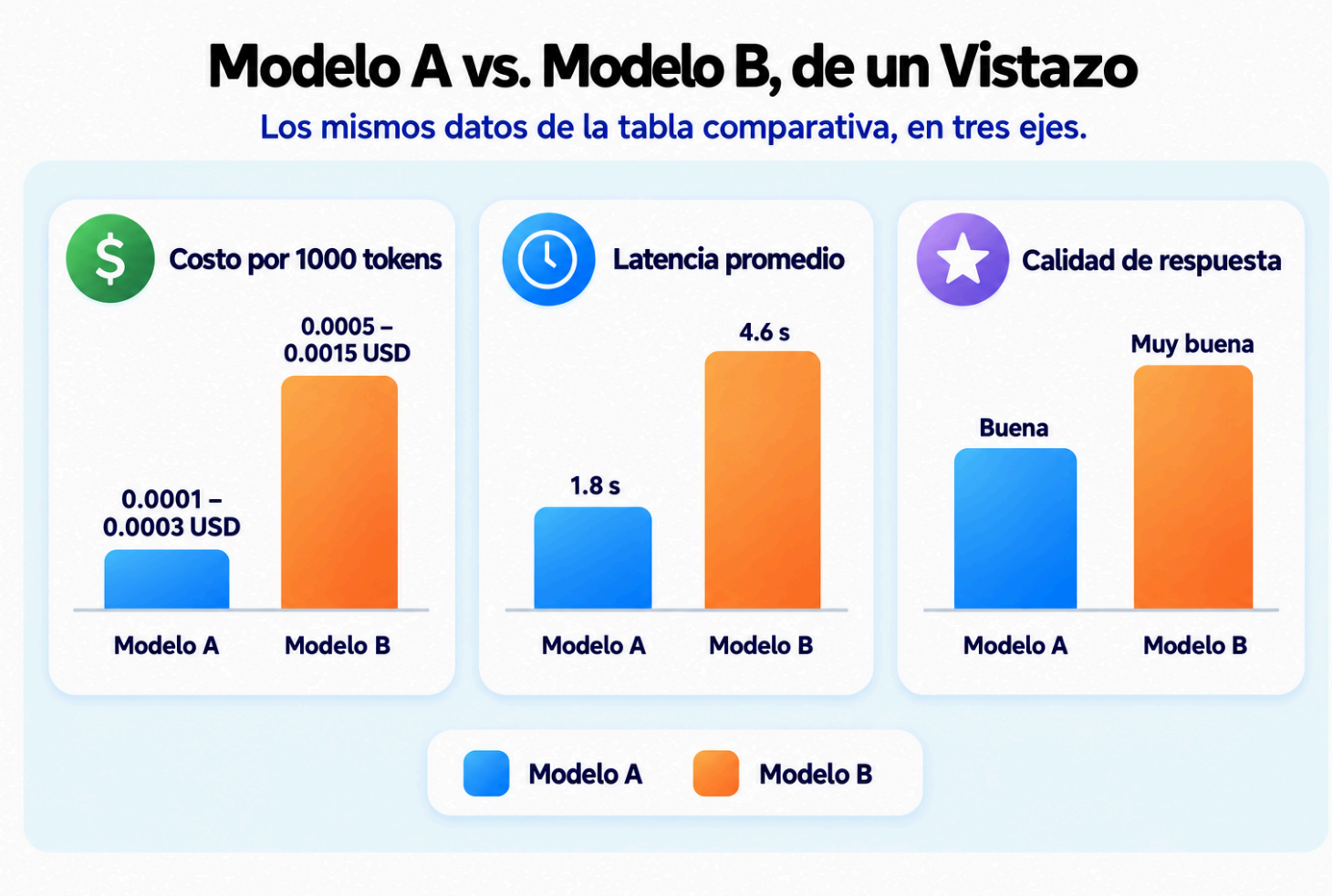


📊 Un ejemplo de tabla ya completa

Para que la estructura anterior deje de ser abstracta, así se vería esa misma tabla ya completa, con datc de ejemplo para dos modelos candidatos (los mismos precios ilustrativos que ya viste en la lecció anterior):

Criterio	Modelo A	Modelo B
Costo por 1000 tokens de entrada	0,0001 USD	0,0005 USD
Costo por 1000 tokens de salida	0,0003 USD	0,0015 USD
Latencia promedio observada	1,8 segundos	4,6 segundos
Calidad de respuesta para tu caso concreto	buena: cubre lo esencial en las 3 preguntas de prueba	muy buena: más detalle y matices en las mismas 3 respuestas

De nuevo: estos números son inventados para el ejemplo, no precios reales de ningún proveedor. Lo que importa es cómo se lee la tabla ya completa, no los valores específicos.



🎯 Cómo se usa la tabla para decidir, no solo para llenarla

Una tabla completa no elige el modelo por ti: la elección sigue siendo tuya, pero ahora apoyada en datc concretos en vez de en una preferencia sin justificar.

La forma de usarla es preguntarte qué criterio pesa más para tu caso específico.

> **Ejemplo**
>
> Elegir según lo que realmente necesita tu aplicación. La forma de usarla es preguntarte qué criterio pesa más para tu caso específico.

Tomemos el ejemplo anterior. El **Modelo B** cuesta cinco veces más que el **Modelo A**, y tarda más del doble en responder, a cambio de respuestas con algo más de detalle.

🤖 Si es un asistente simple

Si tu miniaplicación es un asistente simple, donde **"cubre lo esencial"** ya es suficiente para lo que el usuario necesita, el **Modelo A** es, con estos datos, la elección más razonable: la diferencia de calidad no justifica pagar cinco veces más ni esperar el doble.

🎯 Si el detalle adicional es un requisito real

Si en cambio el detalle adicional del Modelo B fuera un requisito real de tu caso (no un extra agradable, sino algo que tu aplicación necesita para cumplir su propósito), esa misma diferencia de costo y latencia podría valer la pena.

🧪 Si es un prototipo

Si tu **miniaplicación** es un prototipo que vas a correr pocas veces mientras la desarrollas, el costo probablemente pesa poco y la calidad de respuesta pesa mucho.

⚡ Si el usuario espera una respuesta inmediata

Si en cambio estuvieras construyendo algo que un usuario real espera en el momento, la latencia empieza a pesar tanto como la calidad, y un modelo levemente menos preciso pero notablemente más rápido puede ser, en conjunto, la mejor elección.

No hay una fórmula única que combine los tres criterios en un solo número: lo que sí hay es l obligación de mirarlos todos, con datos reales, antes de decidir, y de poder explicar después por qu elegiste el modelo que elegiste, no solo cuál elegiste.

Siguiente lección

Bloque 8 · Decisión de arquitectura

Con tu tabla comparativa completa y tu decisión ya tomada, en la siguiente lección, la actividad práctica que cierra esta unidad, vas a documentar formalmente esa decisión en tu repositorio, con la tabla como evidencia de que la elección tiene respaldo, no es solo una preferencia.

Última modificación: jueves, 24 de septiembre de 2026, 16:32